From quality assurance to quality engineering: a multi-agent, local-first approach for agentic DevOps

Student: João Monteiro

Supervisor: Ilídio Oliveira

Co-supervisors: Sérgio Freire, Pedro Lobo

Summary: Software development, and quality engineering in particular, is undergoing a major shift: LLMs and agentic frameworks are making these practices increasingly automated. However, current cloud-based solutions present drawbacks for industrial adoption: high operational costs, data-privacy concerns and latency. This work addresses these limitations through local-first and hybrid solutions, focused specifically on Quality Engineering.

State of the Art

Solutions have moved from prompting to fine-tuning to agents: 68% of the solutions in a 2025 survey are agent-based

1. Multi-agent systems

Role-based agents across the software lifecycle (survey of 71 studies). MetaGPT and ChatDev lead, both on GPT models

2. Test generation

Generate tests, run them, keep only those that pass. 70% coverage vs. 51% for a classic tool

3. Static analysis & vulnerabilities

The LLM guides CodeQL and filters its alerts: 55 vs. 27 of 120 vulnerabilities found

4. Small models & tool calling

They often break the tool-call format. Two fixes: fine-tuning or guardrails.

What I take from the literature

- **Pair each agent with a tool:** LLMs plus deterministic tools beat either alone
- **Verify outside the model:** keep only tests that run and pass, and only findings a tool result backs up
- **Guardrails around small models:** answers validated against a format, re-asked when invalid, items referred to by id
- **Small models handle narrow steps but judge poorly:** small local models for the narrow steps, a larger model to review their verdicts

Experiments, Challenges and Open Gaps

Hands-on and Early Experiments

- **Test site with planted link and accessibility defects**
- **Site crawler + orchestrator:** Gemini reviews small local workers (qwen3:4b, CPU only)
- **Links:** all 16 planted problems found, 1 false alarm
- **Accessibility:** qwen3:4b found 8 of 11 defects, Gemini 11 of 11

Challenge

On a CPU, runtime depends on how much the model writes (~7 tokens/s with qwen3:4b)

Open Gaps

- **One local-first pipeline for test generation, static analysis and vulnerability detection**
- **Small models working with a larger reviewer, untested in the work I found**